

ACADEMY

Pulso TransMi — Guía operativa de submissions

Proyecto 1 de MLOps · GitHub Actions, entregas y reentrenamiento

DOCUMENTO	CURSO	FECHA	USO
Material académico	MLOps · Ciencia de Datos	21 de septiembre de 2026	Uso académico
INSTITUCIÓN	UNIDAD ACADÉMICA	DOCENTE	
Universidad Externado de Colombia	Departamento de Matemáticas	Julián Zuluaga · Docente	

Resumen ejecutivo

La regla central

Pulso TransMi no pide ejecutar un modelo manualmente a una hora exacta. Pide construir un sistema que **despierte con frecuencia, consulte el estado oficial, actúe solo cuando corresponda y conserve evidencia**.

Automatiza la vigilancia; no multipliques las entregas.

La recomendación es ejecutar GitHub Actions cada 10 minutos. En cada ejecución el pipeline consulta si existe un ciclo abierto. Si ya entregó ese ciclo, termina sin volver a enviar. Si no hay ciclo, también termina correctamente.

La API es la autoridad sobre el ciclo, sus targets y el cierre. El cron solo despierta el proceso.

El contrato operativo

El contrato en seis cifras

Concepto	Valor oficial
Granularidad de las observaciones	15 minutos virtuales
Publicación de datos nuevos	2 timestamps cada 30 minutos reales
Apertura de un ciclo	1 vez por hora real
Ventana para entregar	25 minutos reales
Targets por ciclo ordinario	48 valores
Intentos válidos máximos	3 por ciclo

Qué debe quedar funcionando

- Una memoria incremental de datos en Supabase.
- Un modelo promovido, reproducible y disponible para inferencia.
- Un workflow de GitHub Actions que consulta, predice y envía.
- Un recibo por submission para impedir duplicados y facilitar auditoría.
- Una rutina separada para evaluar, detectar drift y reentrenar.

Tres decisiones que simplifican el proyecto

1. **Consultar antes de actuar.** La API define si hay ciclo y qué targets espera.
2. **Persistir antes de avanzar.** Cursor, recibo y versión del modelo quedan registrados.
3. **Separar operación de aprendizaje.** Inferencia frecuente; entrenamiento deliberado.

El reloj de una hora



Reloj operativo: un batch de 48 predicciones se entrega dentro de una ventana de 25 minutos.

Fuente: Elaboración editorial a partir del contrato Pulso TransMi v0.6.0 (2026-09-21).

Al comienzo de la hora se abre un ciclo. Durante 25 minutos se reciben submissions. A los 30 minutos aparece el siguiente lote de observaciones y, cuando todos los targets ya tienen realidad observada, el sistema puede evaluar las predicciones.

Un ciclo, un batch completo

Cada ciclo ordinario solicita:

```
12 estaciones × 4 horizontes (+15, +30, +45, +60) = 48 predicciones
```

Las 48 predicciones se calculan y se envían juntas. **No son cuatro entregas distintas.** El endpoint `GET /v1/forecast-cycles/current` devuelve la lista exacta de parejas `station_id` + `target_at`, además de `cycle_id`, `data_cutoff` y `closes_at`.

Qué significa cada instante

Momento	Qué ocurre	Qué hace el equipo
Minuto 00	Se abre el ciclo	Sincroniza, consulta targets, infiere y entrega
Minuto 25	Cierra la ventana	Ya no intenta corregir ni enviar tarde
Minuto 30	Se liberan datos nuevos	Actualiza Supabase y métricas disponibles
Minuto 60	Comienza el siguiente ciclo	Repite el loop con el nuevo <code>cycle_id</code>

La hora local del computador no se usa para inventar ciclos ni timestamps. Los campos de la API son la fuente de verdad.

Arquitectura mínima del estudiante

El proyecto separa responsabilidades para que cada falla sea visible y recuperable.

Componente	Responsabilidad	No debe hacer
API Pulso TransMi	Publicar realidad, ciclos, recibos y leaderboard	Guardar el modelo del estudiante
Supabase Postgres	Conservar observaciones, cursores, predicciones y evaluaciones	Exponer la API key públicamente
Supabase Storage o GitHub Release	Guardar artefactos promovidos e inmutables	Sobrescribir silenciosamente el champion
GitHub Actions	Orquestar sincronización, inferencia, entrega y entrenamiento	Reentrenar en cada despertar
Repositorio GitHub	Versionar código, esquema, tests y decisiones	Contener secretos
Vercel — bono	Mostrar accuracy, drift, cobertura y posición	Ejecutar el entrenamiento principal

Dos workflows, dos propósitos

Inferencia y submission. Corre cada 10 minutos, es liviano y solo envía cuando hay un ciclo nuevo abierto.

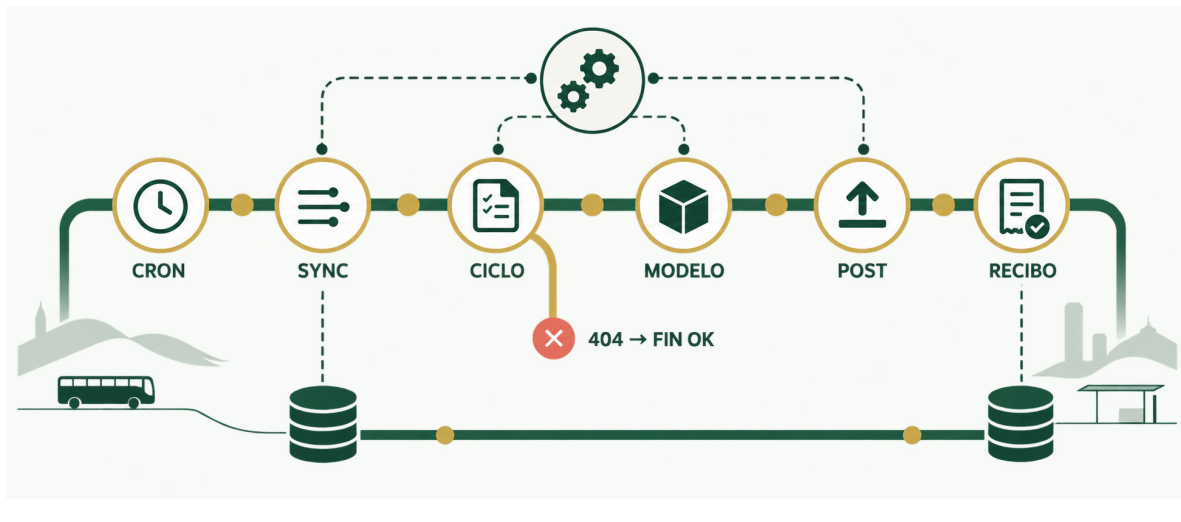
Entrenamiento y promoción. Corre en otro workflow, por decisión manual, programación diaria o una señal de drift. Compara candidatos y solo reemplaza al champion cuando existe evidencia temporal de mejora.

Esta separación evita gastar minutos gratuitos entrenando innecesariamente y evita que un fallo de entrenamiento bloquee una entrega.

Secretos

La API key se guarda como secreto del repositorio con el nombre `PULSO_API_KEY`. No se imprime en logs, no se sube al código, no se guarda en una tabla consultable por el frontend y no se expone como variable pública de Vercel.

El loop de GitHub Actions



El workflow despierta, consulta y decide; no envía nuevamente un ciclo que ya tiene recibo.

Fuente: Elaboración editorial a partir del flujo estudiantil Pulso TransMi (2026-09-21).

Decisión paso a paso

1. **Despertar.** El cron ejecuta el workflow cada 10 minutos.
2. **Sincronizar.** Consume `/v1/stream/observations` desde el último cursor confirmado y guarda de forma idempotente.
3. **Consultar.** Solicita `/v1/forecast-cycles/current`.
4. **Salir si no corresponde.** Un `404 no_open_cycle` es un resultado normal: finaliza en verde.
5. **Evitar duplicados.** Si Supabase ya contiene un recibo para ese `cycle_id` y versión del modelo, finaliza.
6. **Inferir.** Carga el champion y genera exactamente los targets solicitados.
7. **Validar.** Comprueba cantidad, unicidad, timestamps, estaciones y valores finitos no negativos.
8. **Enviar.** Publica el batch completo con API key e `Idempotency-Key` estable.
9. **Guardar recibo.** Persiste `submission_id`, intento, hash, modelo, commit y hora de aceptación.

Por qué cada 10 minutos

La ventana dura 25 minutos y los jobs programados pueden retrasarse. Tres oportunidades aproximadas dentro de la ventana hacen el sistema más tolerante. La protección contra duplicados debe estar en la lógica del estudiante: **despertar tres veces no significa enviar tres veces**.

Regla de idempotencia

Para el mismo ciclo y el mismo contenido se reutiliza la misma llave. Si la red falla después del POST, se reintenta con esa llave; no se crea una nueva entrega a ciegas.

Workflow recomendado

El archivo puede llamarse `.github/workflows/predict.yml`. GitHub interpreta el cron en UTC y los workflows programados se ejecutan desde la rama por defecto.

```
name: pulso-transmi-predict

on:
  schedule:
    - cron: "*/10 * * * *"
  workflow_dispatch:

concurrency:
  group: pulso-transmi-predict
  cancel-in-progress: false

jobs:
  predict:
    runs-on: ubuntu-latest
    timeout-minutes: 8
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements.txt
      - name: Sincronizar, inferir y entregar
        env:
          PULSO_API_KEY: ${ secrets.PULSO_API_KEY }
          SUPABASE_URL: ${ secrets.SUPABASE_URL }
          SUPABASE_SERVICE_KEY: ${ secrets.SUPABASE_SERVICE_KEY }
        run: python -m src.pipeline.submit_current_cycle
```

Guardrails del workflow

- `concurrency` evita dos ejecuciones simultáneas del mismo pipeline.
- `timeout-minutes` impide consumir tiempo indefinidamente.
- El job devuelve código `0` cuando no hay ciclo o cuando ese ciclo ya fue entregado.
- Los secretos se inyectan únicamente en el step que los necesita.
- Los logs muestran IDs, estados y cantidades; nunca credenciales ni payloads sensibles completos.
- `workflow_dispatch` permite una prueba manual controlada sin alterar el cron.

GitHub permite intervalos programados mínimos de cinco minutos, pero advierte que un job puede retrasarse en momentos de alta carga. Por eso el workflow consulta la API en lugar de depender de una ejecución exacta al minuto.

El programa que ejecuta el workflow

La implementación puede organizarse distinto, pero debe conservar esta lógica:

```
sync_observations_from_saved_cursor()

cycle = get_current_cycle()
if cycle is None:                # 404: no hay ciclo abierto
    exit_successfully()

model = load_promoted_model()
if receipt_exists(cycle.id, model.version):
    exit_successfully()

rows = build_features_as_of(cycle.data_cutoff, cycle.targets)
predictions = model.predict(rows)
validate_exact_targets(predictions, cycle.targets)

key = stable_key(cycle.id, model.version, predictions)
receipt = post_submission(
    cycle_id=cycle.id,
    predictions=predictions,
    idempotency_key=key,
    model_metadata=model.metadata,
)
save_receipt(receipt)
```

Datos que acompañan al modelo

Una entrega debe poder explicarse después. Conserve, como mínimo:

- `model_version` y ubicación del artefacto;
- commit de Git utilizado;
- fecha de entrenamiento y último dato usado;
- variables y transformación principales;
- `cycle_id` , `data_cutoff` y hash del batch;
- `submission_id` , intento y hora de aceptación.

El nombre del estudiante no se envía como guardrail. La identidad se deriva de la API key personal.

Qué hacer ante cada respuesta

Respuesta	Interpretación	Acción del pipeline
201	Submission nueva aceptada	Guardar recibo y terminar
200	Repetición idempotente aceptada	Guardar el mismo recibo y terminar
401	API key inválida o ausente	Fallar, revisar el secret y rotar si aplica
404 no_open_cycle	No hay ciclo vigente	Terminar correctamente; no alertar
409	Ciclo cerrado, conflicto o límite de intentos	No reintentar a ciegas; inspeccionar el código
422	El batch viola el contrato	Corregir ensamblaje; no culpar al modelo
429	Exceso de solicitudes	Aplicar backoff y reutilizar la misma llave
5xx o timeout	Falla transitoria	Reintentar con backoff y la misma llave

Checklist antes del POST

- El `cycle_id` viene del endpoint actual.
- Las parejas `station_id` + `target_at` coinciden exactamente con los targets.
- No existen duplicados ni targets adicionales.
- Todos los valores son números finitos y mayores o iguales a cero.
- El batch tiene la cantidad indicada por `expected_predictions`.
- La inferencia solo usa información hasta `data_cutoff`.
- Aún no existe un recibo local para el ciclo y versión.
- La ventana sigue abierta.

La API acepta o rechaza el batch completo. Esta atomicidad evita rankings contruidos con coberturas distintas.

Accuracy, drift y reentrenamiento

La entrega aceptada demuestra que el sistema operó; no que el modelo acertó. Cuando aparece el valor real, la plataforma calcula por estación:

```
WAPE = suma(|real - predicción|) / suma(real)
Accuracy = 100 × max(0, 1 - WAPE)
```

La accuracy oficial promedia las accuracies de las estaciones. La cobertura indica cuántos targets esperados fueron evaluables y debe mantenerse idealmente por encima de 95 %.

Cuándo revisar el modelo

No reentrene por reflejo ante un único error. Investigue cuando se combinen señales como:

- caída persistente de la accuracy `rolling_24h` ;
- degradación concentrada en varias estaciones u horizontes;
- cambio de distribución en variables de entrada;
- crecimiento del error frente a un baseline reciente;
- suficientes observaciones nuevas para una validación temporal útil.

Flujo de promoción

1. Entrenar un candidato con una ventana temporal reproducible.
2. Compararlo contra el champion y contra baselines.
3. Verificar estabilidad por estación y horizonte, no solo el promedio.
4. Registrar experimento, datos, parámetros y métricas.
5. Publicar el artefacto con una versión nueva.
6. Cambiar el puntero de champion.
7. Mantener disponible la versión anterior para rollback.

El workflow de inferencia siempre carga una versión promovida. Nunca toma automáticamente “el último archivo entrenado”.

Próximos pasos, bono visual y criterios de terminación

El dashboard desplegado en Vercel es un bono cuando ayuda a entender y operar el sistema. Debe leer datos derivados desde Supabase y nunca exponer secretos.

Paneles con valor real

Panel	Pregunta que responde
Carrera del leaderboard	¿Cómo cambia la posición a lo largo del tiempo?
Accuracy acumulada y rolling 24 h	¿El desempeño histórico oculta una degradación reciente?
Cobertura	¿Estamos entregando y siendo evaluados de forma consistente?
Error por estación y horizonte	¿Dónde falla el modelo?
Drift	¿La realidad reciente se aleja de lo aprendido?
Estado operativo	¿Cuándo corrió collector, inferencia y evaluación por última vez?

El proyecto está listo cuando...

- [] El histórico y el stream se guardan sin duplicados.
- [] El cursor solo avanza después de una escritura confirmada.
- [] El champion tiene versión, metadata y ubicación estable.
- [] El cron corre cada 10 minutos y también admite ejecución manual.
- [] Un `404` finaliza en verde.
- [] Un ciclo ya entregado no genera otro POST.
- [] El batch reproduce exactamente los targets de la API.
- [] Los reintentos reutilizan la llave de idempotencia.
- [] El recibo queda persistido y relacionado con modelo y commit.
- [] Entrenamiento e inferencia viven en workflows separados.
- [] Accuracy, cobertura y drift se pueden explicar con evidencia.

La competencia no premia al script que corre una vez.

Premia al sistema que observa, decide, entrega, aprende del error y vuelve a operar de forma confiable cuando la ciudad cambia.

Fuentes y referencias

Documentación del proyecto

- [Repositorio público Pulso TransMi](#)
- [Contrato técnico de la API](#)
- [Cliente estudiantil y loop MLOps](#)
- [Primera predicción](#)
- [Portal y gestión de API key](#)

GitHub Actions

- [Workflow syntax for GitHub Actions](#)
- [Events that trigger workflows](#)
- [Secrets](#)

Nota de vigencia

Esta guía explica el contrato operativo vigente el **21 de septiembre de 2026**. Si la API y este documento difieren, prevalecen el endpoint de ciclo actual y el contrato técnico publicado en la versión vigente del repositorio.